

Distributed Semantic Search Engine with RAG over Wikipedia

Project Proposal — Big Data Course Project

Date: September 2026 **Target hardware:** 100+ CPU cores · 1× NVIDIA L4 (24 GB VRAM) · 512 GB RAM

Corpus: Wikipedia webpage dump, Parquet format

1. Overview

Extends the existing PDF-based semantic search + RAG system to a distributed, single-node, multi-core/single-GPU architecture ingesting the full Wikipedia corpus from Parquet. Core pipeline is unchanged in spirit (embed → index → retrieve → rerank → generate), but re-engineered for a multi-million-document corpus, with CPU and GPU each doing only the work they're best at: **CPU for parsing/chunking/index search, GPU exclusively for embedding + reranking inference.**

2. Assumptions

These drive every number below — correct them against your real file before trusting the estimates.

#	Assumption	Why it matters
A1	Parquet has one row per article: <code>[id, title, url, text]</code> (± <code>[categories]</code> , <code>[revision_id]</code>)	Standard shape for Wikipedia-dump-style parquet exports
A2	Full English Wikipedia ≈ 6.5M articles	Verify with a row count on your actual file
A3	Average usable body text ≈ 500–600 tokens/article (heavy stub tail)	Drives the chunk-count estimate in §4.1
A4	Single node, single GPU — multiprocessing/Ray for local parallelism, not multi-node distribution	Matches your hardware, not a real cluster
A5	Embedding model: small/fast/open (e.g. <code>bge-small-en-v1.5</code> or <code>all-MiniLM-L6-v2</code> , 384-dim) as default	Keeps FAISS memory low and leaves GPU headroom for the reranker

If your parquet stores raw wikitext instead of clean text, add an explicit markup-stripping step before chunking (`mwparserfromhell` or regex) — called out in §6.

3. Objectives

Ingest the full Wikipedia parquet corpus end-to-end without OOM or GPU starvation.

Section/heading-aware chunking (carried over from the PDF version), adapted to wiki structure.

A FAISS index that fits comfortably in 512 GB RAM with a tunable recall/latency knob.
Full RAG pipeline — retrieve → rerank → generate — with query logging to Postgres for offline eval.
A resource-aware distributed ingestion design as the actual "Big Data" contribution — this is what should carry marks over a toy semantic-search demo.

4. Capacity Planning

4.1 Corpus → chunk estimate

	
N_docs	≈ 6,500,000 (verify: row count of your parquet)
avg_chunks/doc	≈ 3 (256-token chunks, heading-aware, mostly-short articles)
N_chunks	≈ N_docs × 3 ≈ 19–20,000,000

Recompute the moment you have the real row count — everything downstream scales off it.

4.2 GPU embedding throughput (L4, 24 GB)

Small model (384-dim), FP16, batch 256, seq_len ≤256 tokens:

	
Estimated throughput:	~2,000–3,500 chunks/sec (benchmark on your box — varies by tokenizer overhead)
Full-corpus embed time:	20M / 2,500 ≈ 8,000s ≈ 2.2h pure compute
+ I/O + CPU-feeding overhead (~1.5–2×)	≈ 3.5–4.5h realistic — a single overnight batch job

A bigger model (768-dim, e.g. bge-base/e5-base) roughly halves throughput and doubles index memory — only worth it if a quick recall ablation (§12) shows it earns its keep.

4.3 GPU VRAM budget (24 GB L4)

Component	Approx VRAM	Notes
Embedding model (fp16, batch 256)	~4–5 GB	weights <1GB, rest is activations
Cross-encoder reranker	~4–5 GB	only active at query time
Headroom	~14 GB	keep generation off-GPU so ingestion and serving never fight over VRAM

Decision: don't run a local LLM on the same GPU as the embed/rerank path. Use an API LLM for generation — keeps the GPU 100% dedicated to retrieval throughput, which is the real bottleneck at this scale. For a fully offline demo, note a quantized 7–8B model (int4, ~5–6GB) as a fallback in §9, loaded only for the demo, never during ingestion.

4.4 FAISS index memory (CPU-resident, not GPU)

20M vectors × 384 dims:



IVF-Flat (fp32): $20\text{M} \times 384 \times 4\text{B} \approx 30.7\text{ GB}$

IVF-Flat (fp16): $\approx 15.4\text{ GB}$

IVF-PQ (m=48, 8-bit): $20\text{M} \times 48\text{B} \approx 0.96\text{ GB}$ (+ codebooks, negligible)

With 512 GB RAM, **you don't actually need PQ compression** — IVF-Flat fits easily and gives exact distances within probed lists (better recall than PQ). Recommendation:

Primary index: IVF-Flat, $n_{\text{list}} \approx 32,768$ (rule of thumb $n_{\text{list}} \in [4\sqrt{N}, 16\sqrt{N}]$; $V_{20\text{M}} \approx 4,472$), n_{probe} tuned from a recall/latency sweep (start 8–16).

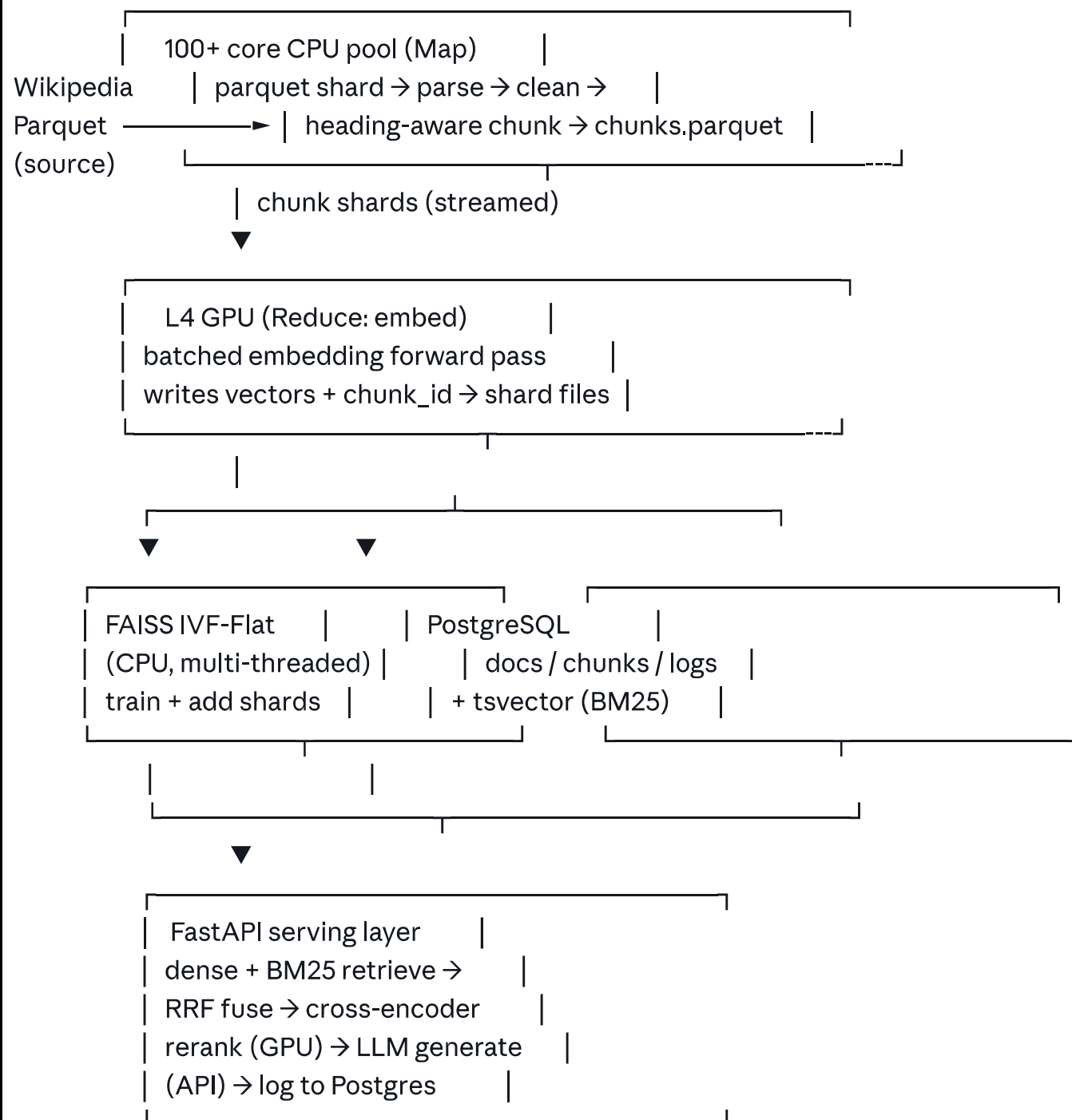
Secondary/ablation: build an IVF-PQ variant of the same data and report recall-vs-memory-vs-latency — a strong evaluation section even though you don't strictly need the compression (§12).

4.5 Postgres sizing

20M chunk rows × (~1.2 KB text + ~150 B metadata) $\approx$ **25–30 GB** — trivial on this box. Store chunk text directly in Postgres (not just pointers), so you can add a `tsvector` column and get hybrid dense+BM25 retrieval essentially for free (§8).

5. High-Level Architecture





6. Ingestion & Chunking (Map stage — 100+ cores)

Split source parquet into ~100–200 row-group shards (`pyarrow.parquet.ParquetFile.iter_batches`, or pre-split files).

`multiprocessing.Pool(processes=~90)` — leave headroom for OS/GPU-feeder threads. One shard per worker. Per worker: strip residual markup/HTML → detect headings (regex on `== Heading ==`, or use a `sections` column if already structured) → heading-aware split into ~256-token chunks with ~15–20% overlap → emit `chunks.parquet` shard with `chunk_id, doc_id, title, url, section_path, text, token_count`.

This stage is likely I/O-bound before it's CPU-bound at this core count — profile on a 100k-doc subset before assuming linear scaling to 100 cores.

7. Embedding (Reduce stage — GPU)

A single streaming consumer reads `chunks.parquet` shards as they land, batches to 256, runs the

embedding model in fp16 (PyTorch, or ONNX Runtime for extra throughput if time allows).

Write `(chunk_id, vector)` shard-by-shard rather than holding all 20M vectors in memory — stream directly into FAISS `add_with_ids` once trained.

Train the IVF quantizer once, on a random sample (~1M chunks), before the full pass; `add` the rest incrementally per shard as embeddings complete.

8. Retrieval: Hybrid Dense + Sparse

Since Postgres already holds chunk text (§4.5), add a `tsvector` GIN index for BM25-style keyword search alongside FAISS dense retrieval:

FAISS IVF-Flat top-50 (dense)

Postgres full-text top-50 (sparse)

Reciprocal Rank Fusion to merge into ~50 candidates

Cross-encoder rerank (GPU) → top 5–8

Pass top passages + provenance to the LLM for generation

Optional relative to the original PDF project, cheap to add here, and meaningfully improves recall on Wikipedia’s mix of entity-lookup vs. conceptual queries — worth stating explicitly as an enhancement over the PDF version.

9. RAG Generation

Default: call an LLM API with the reranked passages + inline citations back to `(title, url, section_path)` — matches the provenance tracking already logged.

Offline fallback: a quantized 7–8B local model (int4, vLLM or llama.cpp), ~5–6 GB VRAM, loaded only for the demo — *after* ingestion finishes, so it never competes with embedding/rerank for VRAM.

10. Storage Schema (PostgreSQL)



sql

```

CREATE TABLE documents (
doc_id    BIGINT PRIMARY KEY,
title     TEXT,
url       TEXT,
source_shard INT
);

CREATE TABLE chunks (
chunk_id  BIGINT PRIMARY KEY,
doc_id    BIGINT REFERENCES documents(doc_id),
section_path TEXT,
text      TEXT,
token_count INT,
faiss_id  BIGINT,      -- maps to FAISS add_with_ids
text_tsv  TSVECTOR     -- BM25 hybrid search
);

CREATE INDEX chunks_tsv_idx ON chunks USING GIN (text_tsv);

CREATE TABLE query_logs (
query_id  BIGSERIAL PRIMARY KEY,
query_text TEXT,
retrieved BIGINT[],
reranked  BIGINT[],
latency_ms INT,
created_at TIMESTAMPTZ DEFAULT now()
);

```

11. API Layer (FastAPI)

Endpoint	Purpose
<code>POST /search</code>	dense+hybrid retrieval only, returns ranked chunks
<code>POST /rag/answer</code>	full pipeline: retrieve → rerank → generate
<code>GET /ingest/status</code>	ingestion progress (docs processed, chunks embedded, index build %)
<code>GET /health</code> , <code>/metrics</code>	liveness + latency/throughput counters

12. Evaluation Plan

Retrieval quality: Recall@k, MRR, nDCG against a Wikipedia-grounded QA set (NQ-open or TriviaQA are built on Wikipedia passages — reuse rather than hand-labeling from scratch).

Ablation (the Big Data course differentiator): IVF-Flat vs IVF-PQ — recall@10 vs. index memory vs. query latency, swept across `nprobe`.

System metrics: ingestion throughput (chunks/sec), full-corpus build time, query p50/p95 latency, GPU utilization during embed vs. rerank phases.

13. Deployment (Docker Compose)

`ingestion-worker` — CPU-heavy, scaled to core count

`embedding-service` — GPU, exclusive L4 access

`api-service` — FastAPI, loads FAISS index into RAM

`postgres`

(optional) `redis` — cache hot queries

(optional) `prometheus` + `grafana` — throughput/latency dashboards for the demo

14. Suggested Timeline (6 weeks — compress as needed)

Week	Deliverable
1	Parquet inspection, schema lock-in, chunking pipeline on a 100k-doc subset
2	Full Map/Reduce pipeline scaled to 100 cores, embedding throughput benchmarked on L4
3	FAISS IVF-Flat build on full corpus, Postgres schema + hybrid search
4	FastAPI + RAG pipeline, generation via API LLM
5	IVF-PQ ablation, evaluation harness (NQ/TriviaQA), latency tuning
6	Docker Compose packaging, demo, report/writeup

15. Risks & Mitigations

Risk	Mitigation
Full-corpus row-group reads saturate I/O before 100 cores saturate CPU	Profile early on a 100k-doc subset; if I/O-bound, fewer/larger workers beat more workers
GPU idle while CPU chunking backlog builds, or vice versa	Decouple via a shard queue/directory so embedding always has buffered work — avoid a strict two-phase Map-then-Reduce
Wiki markup/table/infobox noise degrades chunk quality	Explicit cleaning step before chunking (§6); spot-check a sample of chunks manually
IVF-Flat build time on 20M vectors longer than expected	Train quantizer on a subsample first; add shards incrementally rather than one giant <code>add()</code> call

16. Tech Stack

Python, FastAPI, FAISS (CPU, multi-threaded), PyTorch (GPU embedding + reranking), PostgreSQL (+ `tsvector`), `multiprocessing`/Ray for the Map stage, Docker/Docker Compose, Linux, sentence-transformers or ONNX Runtime, LLM API for generation.

17. Future Work

Swap in a larger embedding model (768-dim) once the pipeline is validated — re-run the §4 capacity math.
Additional Wikipedia language editions, same pipeline, per-language or merged multilingual index.
Incremental updates (new/edited articles) instead of full re-ingestion.